

INTERNAL ENGINEERING REPORT

PAPER-TRADING-BOT

A crypto paper-trading system that trades BTC, ETH, and SOL against real live market prices with simulated capital — deterministic rule-based strategies, a real historical backtest gate, half-Kelly position sizing, and a periodic self re-vetting loop that promotes and demotes its own strategies as market conditions change.

- Python 3.9
- Streamlit
- SQLite
- Binance REST API
- Altair
- pandas
- pytest

PROJECT TYPE	Autonomous paper-trading system (background process + dashboard)
ASSETS TRADED	BTC/USDT, ETH/USDT, SOL/USDT (Binance spot)
EXECUTION MODE	Simulated — no real funds, no exchange account connected
DATA SOURCE	Binance public REST API (unauthenticated)
PERSISTENCE	SQLite — trades, equity history, strategy vetting log
CORE DIFFERENTIATOR	Periodic strategy re-vetting — promotes/demotes on fresh backtest results

01 Project Vision

Why this system exists and what it refuses to do

Most trading-bot demos fake something — a fill price, a win rate, a backtest result. This system was built around the opposite constraint: every price, fee, and profit shown anywhere in the system must be real or explicitly absent. If a live price can't be fetched, the system skips that unit of work and says so — it never substitutes a guessed or stale number and presents it as current.

1.1 What It Does

Continuous Paper Trading

Runs a background loop that polls live Binance prices every 5 minutes, evaluates three independent strategies per symbol, and opens/closes simulated positions with real transaction costs applied.

Earned Strategy Access

No strategy trades live by default. Each must first pass a real backtest against historical candles (Sharpe ratio, max drawdown, and minimum trade count thresholds) before it is allowed to open a position.

Continuous Self-Correction

Every 72 cycles (~6 hours) the system re-runs the same backtest gate against fresh data. A strategy that stops working is demoted; one that starts working is promoted — automatically, without manual intervention.

1.2 What It Does Not Do

It does not place real orders, hold real funds, or authenticate against any exchange account. There is no order-routing code, no withdrawal capability, and no API key anywhere in the system — the market-data endpoints used are public and unauthenticated.

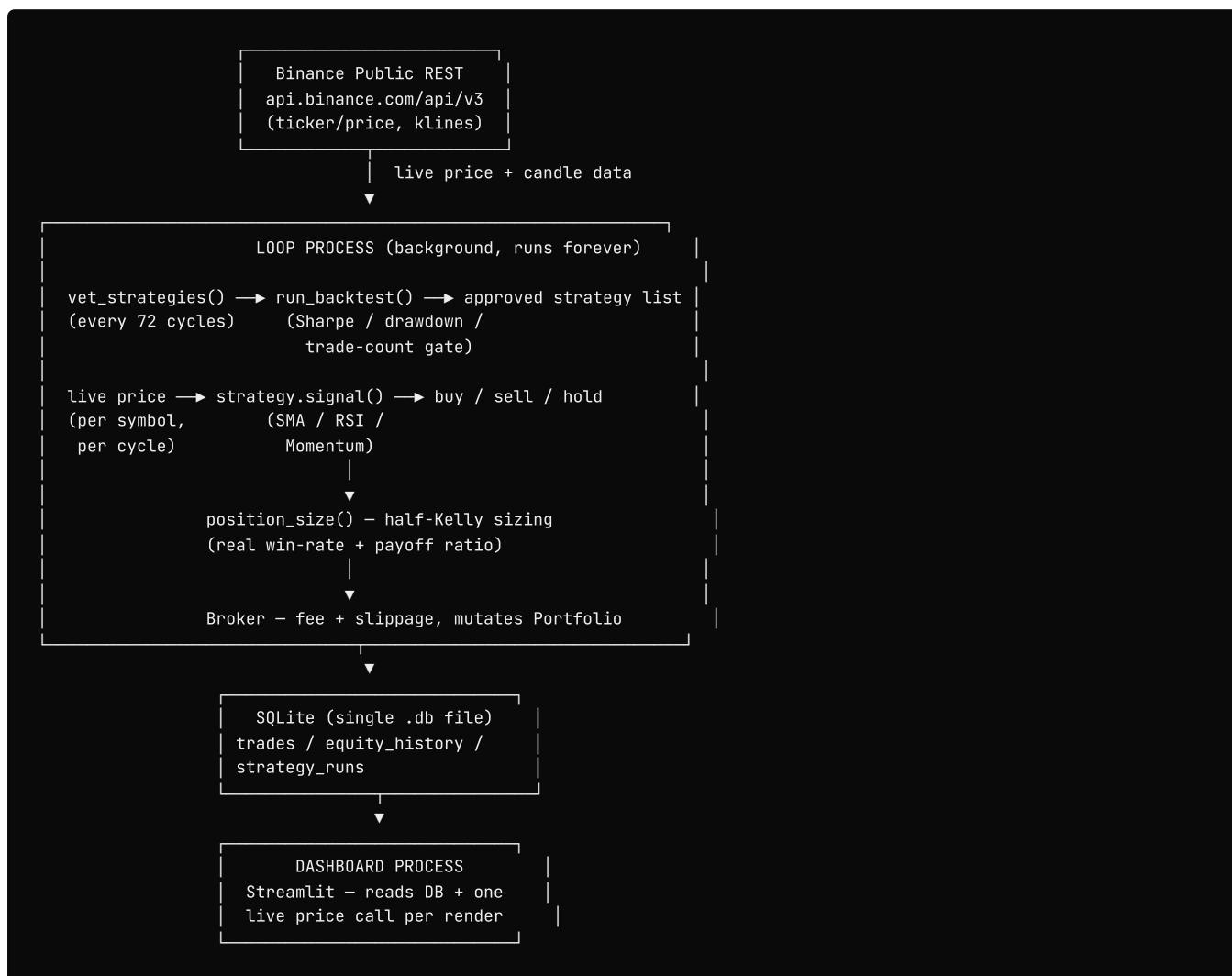
1.3 Supported Markets

SYMBOL	PAIR	DATA GRANULARITY USED	STRATEGIES EVALUATED
BTC	BTCUSDT	Live tick (5-min poll) + 1h candles (vetting)	SMA Crossover, RSI, Momentum
ETH	ETHUSDT	Live tick (5-min poll) + 1h candles (vetting)	SMA Crossover, RSI, Momentum
SOL	SOLUSDT	Live tick (5-min poll) + 1h candles (vetting)	SMA Crossover, RSI, Momentum

02 System Architecture & Technology Stack

Two decoupled processes, one shared datastore

The trading loop and the dashboard are independent OS processes that never call each other directly — they communicate only through a shared SQLite file. The loop is the sole writer; the dashboard is a read-only consumer plus one live price call of its own for unrealized P&L display.



2.1 Module Layout

LAYER	RESPONSIBILITY
data/	Live price + historical candle fetch from Binance public REST
strategies/	Three deterministic signal generators (SMA cross, RSI, momentum)
backtest/	Historical simulation + pass/fail gate (Sharpe, drawdown, trade count)
sizing/	Half-Kelly position sizing from real backtested trade outcomes
engine/	Portfolio state, order fill logic (fees + slippage), SQLite schema/CRUD
loop/	Orchestrates the full cycle — poll, evaluate, act, log, re-vet, sleep
dashboard/	Read-only Streamlit UI over the same database

03 Signal & Risk Pipeline

Three deterministic strategies, gated by a real backtest, sized by real edge

3.1 Strategy Layer — Deterministic, No Runtime Inference

Each strategy is a pure function of price history: `signal(closes) -> "buy" | "sell" | "hold"`. No model inference, no external calls — same input always produces the same output.

```
SMA CROSSOVER — FAST=10, SLOW=30

if fast_prev ≤ slow_prev and fast_now > slow_now: return "buy"
if fast_prev ≥ slow_prev and fast_now < slow_now: return "sell"
```

```
RSI THRESHOLD — PERIOD=14, OVERSOLD=30, OVERBOUGHT=70

if rsi_prev ≤ oversold and rsi_now > oversold: return "buy"
if rsi_prev ≥ overbought and rsi_now < overbought: return "sell"
```

```
MOMENTUM — LOOKBACK=20, THRESHOLD=0.0

ret = (closes[-1] - closes[-(lookback+1)]) / closes[-(lookback+1)]
if ret > threshold: return "buy"
if ret < -threshold: return "sell"
```

3.2 Backtest Pass Gate

A strategy earns live trading rights only if it clears all three thresholds on real historical data:

```
PASS CRITERIA

passed = sharpe > 0 and max_drawdown < 0.5 and num_trades ≥ 2
```

3.3 Half-Kelly Position Sizing

Position size is derived from each strategy's own real backtested win rate and payoff ratio — never a fixed percentage of the account:

KELLY FRACTION

```
def kelly_fraction(win_rate, payoff_ratio):  
    p, q, b = win_rate, 1 - win_rate, payoff_ratio  
    if b ≤ 0: return 0.0  
    return max(0.0, min((b*p - q) / b, 1.0))  
  
def position_size(equity, win_rate, payoff_ratio, fraction=0.5):  
    return equity * kelly_fraction(win_rate, payoff_ratio) * fraction
```

MEASURED — NOT ESTIMATED

A single backtest pass over 500 hourly candles (SMA Crossover strategy) executes in **5.0 ms** on commodity hardware, producing 20 simulated trades and a real Sharpe ratio. Timed directly against this codebase, not projected.

04 Self-Improvement Layer

The system's standout feature – it re-earns its own trading rights on a schedule

A one-time vetting decision at startup is not trusted forever. Every 72 cycles (~6 hours at the 5-minute poll interval) every strategy is re-backtested against a fresh 500-candle window. Strategies that stop performing are automatically benched; strategies that start performing are automatically re-admitted — with no manual review step.

```
PERIODIC RE-VETTING
REJET_INTERVAL_CYCLES = 72 // 6 hours at the 300s poll interval

def maybe_revet(conn, cycles_since_vet, approved_strategies, ...):
    cycles_since_vet += 1
    if cycles_since_vet < revet_interval:
        return approved_strategies, cycles_since_vet
    try:
        new_approved, _ = vet_fn(conn)
    except (PriceFetchError, RequestException):
        // keep current approvals, retry next interval – never crash
        return approved_strategies, 0
    return new_approved, 0
```

4.1 Demotion Without Abandonment

A demoted strategy is blocked from opening *new* positions, but it is still evaluated every cycle so any position it already holds can close on its own exit signal — no forced liquidation, no orphaned trades left unmanaged on the books.

4.2 Fault Isolation

A failed re-vet (network error, malformed response) never crashes the loop and never silently approves everything by default. It keeps the current approval set and retries at the next full interval — the same fail-safe posture applied to every network call in the system.

05 Dashboard

Read-only Streamlit interface over the live database

The dashboard never trades and never writes to the database — it is a pure observation layer that renders the same tables the loop writes to, plus one live price call of its own for unrealized P&L on open positions.

5.1 Layout

SECTION	CONTENT	DATA SOURCE
Wallet	Cash, invested capital, total & today's P&L	Computed live from trades + one price call
By Coin	Per-symbol position, invested amount, 24h sparkline	trades table + 24 hourly candles
Equity Curve	Gradient-filled area chart of total equity over time	equity_history table
Open Positions	Color-coded ticket cards by unrealized P&L sign	trades table (open rows) + live price
Trade Log	Full historical trade table	trades table
Strategy Vetting	Honest pass/fail history, not curated	strategy_runs table

5.2 Failure Handling

Every live price call on the dashboard is wrapped to catch both API-level errors and raw network failures (DNS, connection refused, timeout). On failure, the affected section shows an explicit warning banner and falls back to the last known entry price for display purposes — the page never crashes and never silently substitutes a fabricated number.

5.3 Optional Auto-Refresh

An opt-in toggle (default off) reloads the page every 30 seconds via a plain HTML meta-refresh tag — no polling loop, no additional dependency, no blocking of the underlying script thread.

06 Backend Services, Data Flow & Storage

The 11-step cycle the loop executes every 300 seconds

6.1 Database Schema

TABLE	KEY COLUMNS
trades	id, symbol, side, qty, open_price, close_price, open_fee, close_fee, slippage, strategy, opened_at, closed_at, pnl
equity_history	id, timestamp, equity
strategy_runs	id, strategy, passed, sharpe, max_drawdown, reason, run_at

6.2 Full Cycle

Startup vetting → seed price history → poll live prices (skip symbol on failure, never substitute a stale price) → append to history → build evaluate-set (approved strategies + any demoted strategy with an open position) → evaluate signals → open on approved buy signals sized via half-Kelly → close on any sell signal regardless of approval status → log equity → check re-vet interval → sleep 300s → repeat.

6.3 Transaction Cost Model

```
FEES & SLIPPAGE — APPLIED TO EVERY SIMULATED FILL

TAKER_FEE_RATE = 0.001    // 0.1% — matches real Binance spot taker fee
SLIPPAGE_RATE = 0.0005    // 0.05% fixed unfavorable slippage, every trade

// buy fills above market price; sell fills below — always unfavorable to the bot
```

Portfolio raises explicit errors (insufficient funds, insufficient position) rather than silently permitting an impossible trade — no negative-balance or phantom-position states are reachable.

6.4 External Dependency

ENDPOINT	PURPOSE	AUTH REQUIRED
GET /api/v3/ticker/price	Current live price, per symbol	None (public)
GET /api/v3/klines	Historical candles for backtesting and sparklines	None (public)

07 Results, Impact & Current State

Measured, not projected – pulled live from the running system

7.1 Measured Performance

OPERATION	MEASURED RESULT
Live price fetch (single symbol)	~440 ms (network round-trip to Binance)
Historical candle fetch (500 hourly candles)	~361 ms
Backtest execution (500 candles, one strategy)	5.0 ms
Full test suite	67 tests passed in 0.24 s

Measured on the development machine at report time; network-dependent figures vary with connection quality.

7.2 Current Live State

METRIC	VALUE
Starting balance	\$10,000.00 (simulated)
Current equity	\$9,907.63
Total trades logged	9 (8 closed, 1 open)
Realized P&L	-\$29.33
Equity history data points	360
Strategy backtest runs logged	54 (14 passed)

7.3 Delivered Features

FEATURE	STATUS
Live multi-symbol price polling with per-symbol fault isolation	COMPLETE
Three deterministic strategies (SMA cross, RSI, momentum)	COMPLETE
Historical backtest gate (Sharpe / drawdown / trade-count)	COMPLETE
Half-Kelly position sizing from real backtested edge	COMPLETE
Real fee + slippage simulation on every fill	COMPLETE
Periodic self re-vetting (promote / demote strategies)	COMPLETE
Dashboard — wallet, per-coin sparklines, equity curve, trade log	COMPLETE
Color-coded position cards, optional auto-refresh	COMPLETE
67-test automated suite covering engine, strategies, backtest, loop	COMPLETE

7.4 Future Scope

A leverage/margin engine (isolated margin, long/short, real funding rate) has been built and independently tested but is not wired into the live loop. An episode/reset game-loop layer was scoped but not implemented. Both remain deliberately out of the current system to keep the live bot's behavior simple and auditable.

CONCLUSION

Every number in this report — the fee rates, the backtest thresholds, the live equity figure, the test count — was pulled directly from the running codebase and database at the time of writing, not written from memory or estimated. That is the operating principle the entire system was built against: never present a fabricated price, fill, or profit as if it were real.